

CS 505 Module Four Activity Algorithmic Complexity Template

Complete this template by replacing the bracketed text with the relevant information.

1. Explain the time complexity of the function provided in terms of Big O notation:
The following function takes an array *arr* of size *n* and returns the sum of all elements in the array:

```
int array_sum(int arr[], int n) {  
    int total = 0;  
    for (int i = 0; i < n; i++) {  
        total += arr[i];  
    }  
    return total;  
}
```

Answer: The time complexity of the provided function is $O(n)$. This is because the loop iterates over each item in the array exactly once, resulting in a linear time complexity.

2. Explain the space complexity of the function provided:
The following function takes an array *arr* of size *n* and returns the sum of all elements in the array:

```
int array_sum(int arr[], int n) {  
    int total = 0;  
    for (int i = 0; i < n; i++) {  
        total += arr[i];  
    }  
    return total;  
}
```

Answer: The input array requires $O(n)$ space complexity because it stores *n* elements. However, the function itself uses a constant amount of auxiliary space, resulting in an $O(1)$ space complexity.

3. Explain the space and time complexity of the provided code fragments in Big O notation as m and/or n functions:

A.

```
int sum = 0;
    for (int i = 2; i <= n; i++)
        for (int j = 2; j <= n; j+=10)
            sum += (i+j);
```

Time: $O(n^2)$ – The outer loop executes n times. The inner loop executes $\frac{n}{10}$ times.

$n \times \frac{n}{10} = \frac{n^2}{10}$ – Dropping the constants leaves n^2 which is a quadratic time complexity.

Space: $O(1)$ – The function uses a constant amount of auxiliary space because it only stores 3 variables, regardless of input size.

B.

```
int sum1 = 0;
    for (int i = 20; i <= m*m; i++)
sum1 += i;
    int sum2 = 4;
    for (int j = 12; j <= m; j++)
sum2 += j;
```

Time: $O(m^2)$ – The first loop iterates $m \times m$ times (m^2), resulting in quadratic growth depending on the input size. The second loop iterates m times, resulting in linear growth. When calculating complexity, we keep the highest order term (m^2) resulting in a complexity of $O(m^2)$.

Space: $O(1)$ – The function uses a constant amount of auxiliary space because it only stores a few variables, regardless of input size.

4. Explain the time complexity of the method provided using Big O notation in the worst case. The following method returns true if an array of n integers has two adjacent values that are the same, or else it returns false:

```
public static boolean adjacentDuplicates(int[] a) {
    for (int i = 2; i < a.length+4; i++)
        if (a[i] == a[i-4])
            return true;
    return false;
}
```

Note: As written, the method would throw an `ArrayIndexOutOfBoundsException` because it starts at $i = 2$, which makes $a[i-4]$ use a negative index. The condition $i < a.length + 4$ would also eventually exceed the array bounds. I assumed the intended loop was valid and analyzed the intended linear search behavior.

Answer: The time complexity of the provided method in the worst case is $O(n)$. This is because the loop may need to iterate through the entire array before returning false. Each iteration performs a constant-time comparison, and the number of comparisons grows linearly with the array size, resulting in a time complexity of $O(n)$.

5. Explain the time complexity of the method provided using Big O notation in the best case. The following method returns true if an array of n integers has two adjacent values that are the same, or else it returns false:

```
public static boolean adjacentDuplicates(int[] a) {  
    for (int i = 2; i < a.length+4; i++)  
        if (a[i] == a[i-4])  
            return true;  
    return false;  
}
```

Note: As written, the method would throw an `ArrayIndexOutOfBoundsException` because it starts at $i = 2$, which makes $a[i-4]$ use a negative index. The condition $i < a.length + 4$ would also eventually exceed the array bounds. I assumed the intended loop was valid and analyzed the intended linear search behavior.

Answer: The time complexity of the provided method in the best case is $O(1)$. This is because, in the best case, the method returns true on the first comparison, requiring only a constant number of operations regardless of the array size.

6. Analyze the time and space complexity of the provided linear search algorithm.

```
int linear_search(int arr[], int n, int target) {  
    for (int i = 0; i < n; i++) {  
        if (arr[i] == target) {  
            return i;  
        }  
    }  
    return -1;  
}
```

Time: $O(n)$ – The time complexity of this algorithm is $O(n)$ because, in the worst case, the loop may need to iterate through the entire array before finding the target value or returning -1. The number of operations grows linearly with the size of the array.

Space: $O(1)$ – The function uses a constant amount of auxiliary space because it only stores a few variables, regardless of input size.

7. Determine how many additional operations each algorithm will perform given the constraints for each.

- A. Suppose an algorithm processes n data elements using exactly n^2 operations, where $n = 12$. Calculate how many additional operations the algorithm will perform if the number of data elements is doubled.

Answer:

Original operations = $n^2 = 12^2 = 144$

Operations when the quantity of data is doubled = $n^2 = (12 * 2)^2 = 24^2 = 576$

Additional Operation(s) = $576 - 144 = 432$

Therefore, doubling the input size results in **432 additional operations**.

- B. Suppose an algorithm processes n data elements using exactly $\log_2 n$ operations, where $n = 15$. Calculate how many additional operations the algorithm will perform if the number of data elements is doubled.

Answer:

Original operations = $\log_2 n = \log_2 15 = \sim 4$

Operations when the quantity of data is doubled

$$= \log_2 n = \log_2 (15 * 2) = \log_2 30 = \sim 5$$

Additional Operation(s) = $5 - 4 = 1$

Therefore, doubling the input size results in approximately **1 additional operation**

- C. Suppose an algorithm processes n data elements using exactly 2^n operations, where $n = 20$. Calculate how many additional operations the algorithm will perform if the number of data elements is doubled.

Answer:

Original operations = $2^n = 2^{20} = 1,048,576$

Operations when the quantity of data is doubled

$$= 2^n = 2^{20*2} = 2^{40} = 1,099,511,627,776$$

Additional Operation(s)

$$= 1,099,511,627,776 - 1,048,576 = 1,099,510,579,200$$

Therefore, doubling the input size results in **1,099,510,579,200 additional operations**

8. Prove that the sum of the first n natural numbers, $1 + 2 + 3 + \dots + n$, is equal to $n(n + 1) / 2$ for all positive integers n .

Base Case:

When $n = 1$

$$1 = \frac{1(1+1)}{2} = \frac{1(2)}{2} = \frac{2}{2} = 1$$

Now, assume for some positive integer k :

$$1 + 2 + 3 + \dots + k = \frac{k(k + 1)}{2}$$

Consider $k + 1$:

$$1 + 2 + 3 + \dots + k + (k + 1)$$

$$\frac{k(k + 1)}{2} + (k + 1) = \frac{k(k + 1)}{2} + \frac{2(k + 1)}{2} = \frac{k(k + 1) + 2(k + 1)}{2}$$

$$\text{Factor out } k + 1 = \frac{(k+1)(k+2)}{2} = \frac{(k+1)((k+1)+1)}{2}$$

Since $k + 1$ maintains the form $\frac{n(n+1)}{2}$, the statement remains true for $k + 1$. Therefore, by mathematical induction, the formula is true for all positive integers n .

9. Prove by induction that $10^n - 1$ is divisible by 9 for all non-negative integers.

Deleted: 8

Base Case:

When $n = 0$

$$10^0 - 1 = 1 - 1 = 0$$

$$0 \text{ is divisible by } 9: \frac{0}{9} = 0$$

Now, assume for some non-negative integer k :

$10^k - 1$ is divisible by 9.

This means there exists some integer m such that: $10^k - 1 = 9m$

Consider $k + 1$

$$10^{k+1} - 1 = 10(10^k) - 1 = 10(10^k) - 10 + 9 = 10(10^k - 1) + 9$$

$$\text{Since } 10^k - 1 = 9m: 10(10^k - 1) + 9 = 10(9m) + 9 = 90m + 9 = 9(10m + 1)$$

Since $10m + 1$ is an integer, $9(10m + 1)$ is divisible by 9.

Therefore, $10^{k+1} - 1$ is divisible by 9 for all non-negative integers.

10. Prove by induction that a full binary tree with n internal nodes has exactly $n + 1$ leaves.

Base Case:

$$n = 0$$

If a full binary tree has 0 internal nodes, it consists of only 1 node, which is a leaf:

$$0 + 1 = 1$$

When $n = 0$ the statement is true.

Assume that any full binary tree with k internal nodes has exactly $k + 1$ leaves.

Consider a full binary tree with $k + 1$ internal nodes.

A full binary tree is a tree where every internal node has exactly two children. Starting with a full binary tree with k internal nodes, replace one leaf with an internal node that has two leaf children. This results in:

$$\text{new leaves} = (k + 1) - 1 + 2$$

1 is subtracted since the original leaf is no longer a leaf and now a node, and 2 is added because the new internal node has two leaf children.

$$\text{Simplified: } (k + 1) - 1 + 2 = (k + 1) + 1$$

Since the new tree has $n = k + 1$ internal nodes, $(k + 1) + 1 = n + 1$ which makes the statement true.